

06. Security trong Spring Boot

6.1. Giới thiệu về Spring Security

Spring Security là một framework bảo mật mạnh mẽ và có khả năng tùy biến cao, cung cấp các giải pháp xác thực (authentication) và ủy quyền (authorization) toàn diện cho các ứng dụng Java. Khi tích hợp với Spring Boot, việc cấu hình Spring Security trở nên đơn giản hơn rất nhiều nhờ vào các cấu hình mặc định thông minh và khả năng tự động cấu hình.

Spring Security hoạt động dựa trên một chuỗi các bộ lọc (filter chain) trong Servlet Container, cho phép chặn và xử lý các request trước khi chúng đến được controller. Điều này giúp đảm bảo rằng chỉ những người dùng được xác thực và có quyền phù hợp mới có thể truy cập vào các tài nguyên được bảo vệ.

6.1.1. Các khái niệm cốt lõi

- **Authentication (Xác thực):** Quá trình xác minh danh tính của người dùng (bạn là ai?). Ví dụ: đăng nhập bằng username/password, JWT token, OAuth2, v.v.
- **Authorization (Ủy quyền):** Quá trình xác định xem người dùng đã được xác thực có quyền truy cập vào một tài nguyên cụ thể hay thực hiện một hành động cụ thể hay không (bạn được phép làm gì?).
- **Principal:** Đối tượng đại diện cho người dùng hiện tại trong hệ thống.
- **Authorities/Roles:** Các quyền hoặc vai trò được gán cho người dùng.
- **Security Context:** Chứa thông tin bảo mật của người dùng hiện tại trong suốt quá trình xử lý request.

6.2. Thêm Spring Security vào dự án

Để sử dụng Spring Security, bạn chỉ cần thêm dependency `spring-boot-starter-security` vào file `pom.xml` :

```
XML
```

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
```

Sau khi thêm dependency này, Spring Boot sẽ tự động cấu hình một số thiết lập bảo mật mặc định:

- Tất cả các request HTTP đều yêu cầu xác thực
- Tạo một form đăng nhập mặc định tại `/login`
- Tạo một người dùng mặc định với username `user` và mật khẩu ngẫu nhiên (được in ra console)
- Bảo vệ các endpoint Actuator
- Kích hoạt CSRF protection cho các request POST/PUT/DELETE

6.3. Cấu hình Security cơ bản

6.3.1. In-Memory Authentication

Đây là cách đơn giản nhất để cấu hình xác thực, phù hợp cho việc phát triển và testing:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
```

```

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/public/**", "/api/auth/**").permitAll()
                .requestMatchers("/admin/**").hasRole("ADMIN")
                .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login")
                .defaultSuccessUrl("/dashboard")
                .permitAll()
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
                .logoutSuccessUrl("/login?logout")
                .permitAll()
            )
            .csrf(csrf -> csrf.disable()); // Tắt CSRF cho API

        return http.build();
    }

    @Bean
    public UserDetailsService userDetailsService() {
        UserDetails user = User.builder()
            .username("user")
            .password(passwordEncoder().encode("password"))
            .roles("USER")
            .build();

        UserDetails admin = User.builder()
            .username("admin")
            .password(passwordEncoder().encode("admin123"))
            .roles("ADMIN", "USER")
            .build();

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public PasswordEncoder passwordEncoder() {

```

```
        return new BCryptPasswordEncoder();
    }
}
```

6.3.2. Database Authentication với JPA

Trong ứng dụng thực tế, thông tin người dùng thường được lưu trữ trong cơ sở dữ liệu:

Entity Classes:

Java

```
import javax.persistence.*;
import java.util.Set;

@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String username;

    @Column(nullable = false)
    private String password;

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private boolean enabled = true;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(
        name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id")
    )
    private Set<Role> roles;

    // Constructors, getters, and setters
    public User() {}

    public User(String username, String password, String email) {
```

```

        this.username = username;
        this.password = password;
        this.email = email;
    }

    // Getters and setters...
}

@Entity
@Table(name = "roles")
public class Role {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(unique = true, nullable = false)
    private String name;

    // Constructors, getters, and setters
    public Role() {}

    public Role(String name) {
        this.name = name;
    }

    // Getters and setters...
}

```

Repository:

Java

```

import org.springframework.data.jpa.repository.JpaRepository;
import java.util.Optional;

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByUsername(String username);
    boolean existsByUsername(String username);
    boolean existsByEmail(String email);
}

```

Custom UserDetailsService:

Java

```

import org.springframework.security.core.GrantedAuthority;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.core.userdetails.UsernameNotFoundException;
import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.stream.Collectors;

@Service
public class CustomUserDetailsService implements UserDetailsService {

    private final UserRepository userRepository;

    public CustomUserDetailsService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) throws
UsernameNotFoundException {
        User user = userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not
found: " + username));

        Set<GrantedAuthority> authorities = user.getRoles().stream()
            .map(role -> new SimpleGrantedAuthority("ROLE_" +
role.getName()))
            .collect(Collectors.toSet());

        return new org.springframework.security.core.userdetails.User(
            user.getUsername(),
            user.getPassword(),
            user.isEnabled(),
            true, // accountNonExpired
            true, // credentialsNonExpired
            true, // accountNonLocked
            authorities
        );
    }
}

```

6.4. JWT Authentication

JSON Web Token (JWT) là một chuẩn mở (RFC 7519) để truyền thông tin an toàn giữa các bên dưới dạng JSON object. JWT thường được sử dụng trong các RESTful APIs để xác thực người dùng một cách stateless.

6.4.1. Thêm JWT Dependencies

XML

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.5</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.5</version>
  <scope>runtime</scope>
</dependency>
```

6.4.2. JWT Utility Class

Java

```
import io.jsonwebtoken.*;
import io.jsonwebtoken.security.Keys;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.stereotype.Component;

import java.security.Key;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;
import java.util.function.Function;

@Component
public class JwtUtil {
```

```

@Value("${jwt.secret}")
private String secret;

@Value("${jwt.expiration}")
private Long expiration;

private Key getSigningKey() {
    return Keys.hmacShaKeyFor(secret.getBytes());
}

public String extractUsername(String token) {
    return extractClaim(token, Claims::getSubject);
}

public Date extractExpiration(String token) {
    return extractClaim(token, Claims::getExpiration);
}

public <T> T extractClaim(String token, Function<Claims, T>
claimsResolver) {
    final Claims claims = extractAllClaims(token);
    return claimsResolver.apply(claims);
}

private Claims extractAllClaims(String token) {
    return Jwts.parserBuilder()
        .setSigningKey(getSigningKey())
        .build()
        .parseClaimsJws(token)
        .getBody();
}

private Boolean isTokenExpired(String token) {
    return extractExpiration(token).before(new Date());
}

public String generateToken(UserDetails userDetails) {
    Map<String, Object> claims = new HashMap<>();
    return createToken(claims, userDetails.getUsername());
}

private String createToken(Map<String, Object> claims, String subject) {
    return Jwts.builder()
        .setClaims(claims)
        .setSubject(subject)
        .setIssuedAt(new Date(System.currentTimeMillis()))
        .setExpiration(new Date(System.currentTimeMillis() +

```



```

expiration))
                .signWith(getSigningKey(), SignatureAlgorithm.HS256)
                .compact();
        }

        public Boolean validateToken(String token, UserDetails userDetails) {
            final String username = extractUsername(token);
            return (username.equals(userDetails.getUsername()) &&
!isTokenExpired(token));
        }
    }
}

```

6.4.3. JWT Authentication Filter

Java

```

import
org.springframework.security.authentication.UsernamePasswordAuthenticationTok
en;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import
org.springframework.security.web.authentication.WebAuthenticationDetailsSourc
e;
import org.springframework.stereotype.Component;
import org.springframework.web.filter.OncePerRequestFilter;

import javax.servlet.FilterChain;
import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final UserDetailsService userDetailsService;
    private final JwtUtil jwtUtil;

    public JwtAuthenticationFilter(UserDetailsService userDetailsService,
JwtUtil jwtUtil) {
        this.userDetailsService = userDetailsService;
        this.jwtUtil = jwtUtil;
    }
}

```

```

@Override
protected void doFilterInternal(HttpServletRequest request,
HttpServletRequest response,
                                FilterChain filterChain) throws
ServletException, IOException {

    final String authorizationHeader =
request.getHeader("Authorization");

    String username = null;
    String jwt = null;

    if (authorizationHeader != null &&
authorizationHeader.startsWith("Bearer ")) {
        jwt = authorizationHeader.substring(7);
        try {
            username = jwtUtil.extractUsername(jwt);
        } catch (Exception e) {
            logger.error("Cannot extract username from JWT token", e);
        }
    }

    if (username != null &&
SecurityContextHolder.getContext().getAuthentication() == null) {
        UserDetails userDetails =
this.userDetailsService.loadUserByUsername(username);

        if (jwtUtil.validateToken(jwt, userDetails)) {
            UsernamePasswordAuthenticationToken authToken =
                new UsernamePasswordAuthenticationToken(userDetails,
null, userDetails.getAuthorities());
            authToken.setDetails(new
WebAuthenticationDetailsSource().buildDetails(request));

SecurityContextHolder.getContext().setAuthentication(authToken);
        }
    }

    filterChain.doFilter(request, response);
}
}

```

6.4.4. Authentication Controller

Java

```

import org.springframework.http.ResponseEntity;
import org.springframework.security.authentication.AuthenticationManager;
import org.springframework.security.authentication.BadCredentialsException;
import
org.springframework.security.authentication.UsernamePasswordAuthenticationTok
en;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.web.bind.annotation.*;

@RestController
@RequestMapping("/api/auth")
public class AuthController {

    private final AuthenticationManager authenticationManager;
    private final UserDetailsService userDetailsService;
    private final JwtUtil jwtUtil;

    public AuthController(AuthenticationManager authenticationManager,
                          UserDetailsService userDetailsService,
                          JwtUtil jwtUtil) {
        this.authenticationManager = authenticationManager;
        this.userDetailsService = userDetailsService;
        this.jwtUtil = jwtUtil;
    }

    @PostMapping("/login")
    public ResponseEntity<?> login(@RequestBody LoginRequest loginRequest) {
        try {
            authenticationManager.authenticate(
                new
UsernamePasswordAuthenticationToken(loginRequest.getUsername(),
loginRequest.getPassword())
            );
        } catch (BadCredentialsException e) {
            throw new BadCredentialsException("Invalid credentials", e);
        }

        final UserDetails userDetails =
userDetailsService.loadUserByUsername(loginRequest.getUsername());
        final String jwt = jwtUtil.generateToken(userDetails);

        return ResponseEntity.ok(new JwtResponse(jwt));
    }

    // DTOs
    public static class LoginRequest {

```

```

        private String username;
        private String password;

        // Getters and setters
        public String getUsername() { return username; }
        public void setUsername(String username) { this.username = username;
    }

        public String getPassword() { return password; }
        public void setPassword(String password) { this.password = password;
    }

    }

    public static class JwtResponse {
        private String token;

        public JwtResponse(String token) {
            this.token = token;
        }

        public String getToken() { return token; }
        public void setToken(String token) { this.token = token; }
    }
}

```

6.5. Method-level Security

Bạn có thể sử dụng các annotation để kiểm soát quyền truy cập ở cấp độ phương thức:

Java

```

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.access.prepost.PostAuthorize;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    @PreAuthorize("hasRole('ADMIN')")
    public void deleteProduct(Long id) {
        // Chỉ ADMIN mới có thể xóa sản phẩm
    }

    @PreAuthorize("hasRole('USER') or hasRole('ADMIN')")
    public Product getProduct(Long id) {
        // USER hoặc ADMIN có thể xem sản phẩm
    }
}

```

```

        return productRepository.findById(id).orElse(null);
    }

    @PostAuthorize("returnObject.owner == authentication.name or
hasRole('ADMIN')")
    public Product getProductDetails(Long id) {
        // Chỉ chủ sở hữu hoặc ADMIN mới có thể xem chi tiết
        return productRepository.findById(id).orElse(null);
    }

    @PreAuthorize("#username == authentication.name or hasRole('ADMIN')")
    public void updateUser(String username, User user) {
        // Chỉ chính người dùng đó hoặc ADMIN mới có thể cập nhật
    }
}

```

Để kích hoạt method-level security, thêm annotation `@EnableMethodSecurity` vào configuration class:

```

Java

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true)
public class SecurityConfig {
    // ...
}

```

6.6. Bản chất của Security trong Spring Boot

Bản chất của Security trong Spring Boot là cung cấp một framework bảo mật toàn diện, linh hoạt và dễ cấu hình cho các ứng dụng Java. Nó hoạt động dựa trên nguyên tắc "defense in depth" (phòng thủ nhiều lớp), sử dụng một chuỗi các bộ lọc (filter chain) để chặn và xử lý các request, đảm bảo rằng chỉ những người dùng được xác thực và có quyền phù hợp mới có thể truy cập vào các tài nguyên được bảo vệ. Spring Boot đơn giản hóa việc cấu hình bằng cách cung cấp các giá trị mặc định hợp lý và khả năng tự động cấu hình, đồng thời vẫn cho phép tùy chỉnh sâu khi cần thiết. Điều này giúp các nhà phát triển có thể nhanh chóng thêm các tính năng bảo mật mạnh mẽ vào ứng dụng của họ, từ xác thực đơn giản đến các giải pháp phức tạp như JWT, OAuth2, và SAML, đảm bảo rằng ứng dụng được bảo vệ chống lại các mối đe dọa bảo mật phổ biến.

6.4.5. Cấu hình Spring Security cho JWT Authentication

Để tích hợp JWT vào Spring Security, bạn cần cấu hình `SecurityFilterChain` để thêm `JwtAuthenticationFilter` vào chuỗi lọc và vô hiệu hóa CSRF cũng như quản lý session.

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.authentication.dao.DaoAuthenticationProvider;
import
org.springframework.security.config.annotation.authentication.configuration.A
uthenticationConfiguration;
import
org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.UserDetailsService;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import
org.springframework.security.web.authentication.UsernamePasswordAuthenticatio
nFilter;

@Configuration
@EnableWebSecurity
@EnableMethodSecurity(prePostEnabled = true) // Kích hoạt method-level
security
public class JwtSecurityConfig {

    private final UserDetailsService userDetailsService;
    private final JwtAuthenticationFilter jwtAuthenticationFilter;
    private final JwtAuthEntryPoint jwtAuthEntryPoint; // Để xử lý lỗi xác
thực

    public JwtSecurityConfig(UserDetailsService userDetailsService,
                             JwtAuthenticationFilter jwtAuthenticationFilter,
                             JwtAuthEntryPoint jwtAuthEntryPoint) {
        this.userDetailsService = userDetailsService;
    }
}
```

```

        this.jwtAuthenticationFilter = jwtAuthenticationFilter;
        this.jwtAuthEntryPoint = jwtAuthEntryPoint;
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable()) // Vô hiệu hóa CSRF vì JWT là
stateless
            .exceptionHandling(exception ->
exception.authenticationEntryPoint(jwtAuthEntryPoint)) // Xử lý lỗi xác thực
            .sessionManagement(session ->
session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)) // JWT là
stateless
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/api/auth/**").permitAll() // Cho phép truy
cập các endpoint xác thực
                .requestMatchers("/public/**").permitAll() // Các endpoint
công khai
                .anyRequest().authenticated() // Các request khác yêu cầu xác
thực
            )
            .authenticationProvider(authenticationProvider());

        // Thêm JWT filter trước UsernamePasswordAuthenticationFilter
        http.addFilterBefore(jwtAuthenticationFilter,
UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }

    @Bean
    public DaoAuthenticationProvider authenticationProvider() {
        DaoAuthenticationProvider authProvider = new
DaoAuthenticationProvider();

```

```

        authProvider.setUserDetailsService(userDetailsService);
        authProvider.setPasswordEncoder(passwordEncoder());
        return authProvider;
    }
}

```

JwtAuthEntryPoint (để xử lý lỗi 401 Unauthorized):

Java

```

import org.springframework.security.core.AuthenticationException;
import org.springframework.security.web.AuthenticationEntryPoint;
import org.springframework.stereotype.Component;

import javax.servlet.ServletException;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;

@Component
public class JwtAuthEntryPoint implements AuthenticationEntryPoint {

    @Override
    public void commence(HttpServletRequest request, HttpServletResponse
        response,
                        AuthenticationException authException) throws
        IOException, ServletException {
        response.sendError(HttpServletResponse.SC_UNAUTHORIZED, "Error:
        Unauthorized");
    }
}

```

Với cấu hình này, khi một request đến, `JwtAuthenticationFilter` sẽ kiểm tra JWT token trong header. Nếu token hợp lệ, người dùng sẽ được xác thực và thông tin của họ sẽ được đặt vào `SecurityContext`. Nếu không có token hoặc token không hợp lệ, `JwtAuthEntryPoint` sẽ xử lý lỗi 401 Unauthorized.

6.6. OAuth2/OpenID Connect

OAuth2 là một framework ủy quyền cho phép ứng dụng của bên thứ ba có được quyền truy cập giới hạn vào một dịch vụ HTTP, thay mặt cho chủ sở hữu tài nguyên. OpenID Connect (OIDC) là một lớp xác thực được xây dựng trên OAuth2, cho phép client xác minh danh tính

của người dùng cuối dựa trên xác thực được thực hiện bởi Authorization Server, cũng như lấy thông tin hồ sơ cơ bản về người dùng cuối.

Spring Security cung cấp hỗ trợ mạnh mẽ cho cả OAuth2 Client và OAuth2 Resource Server, giúp tích hợp dễ dàng với các nhà cung cấp OAuth2/OIDC như Google, Okta, Auth0, Keycloak, v.v.

6.6.1. OAuth2 Client (Đăng nhập với bên thứ ba)

Để cho phép người dùng đăng nhập vào ứng dụng của bạn thông qua một nhà cung cấp OAuth2/OIDC (ví dụ: Google, GitHub), bạn cần cấu hình ứng dụng của mình như một OAuth2 Client.

Bước 1: Thêm Dependency

XML

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Bước 2: Cấu hình trong `application.yml`

YAML

```
spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: your-google-client-id
            client-secret: your-google-client-secret
            scope: openid,profile,email
          github:
            client-id: your-github-client-id
            client-secret: your-github-client-secret
        provider:
```

```
google:
  authorization-uri: https://accounts.google.com/o/oauth2/v2/auth
  token-uri: https://oauth2.googleapis.com/token
  user-info-uri: https://openidconnect.googleapis.com/v1/userinfo
  jwk-set-uri: https://www.googleapis.com/oauth2/v3/certs
  user-name-attribute: name
```

- `registration` : Định nghĩa các nhà cung cấp OAuth2 mà ứng dụng của bạn sẽ tích hợp.
- `provider` : Cấu hình chi tiết cho từng nhà cung cấp (thường không cần thiết nếu nhà cung cấp là một trong những nhà cung cấp phổ biến mà Spring Security đã biết).

Bước 3: Cấu hình SecurityFilterChain

Spring Security sẽ tự động tạo một trang đăng nhập với các nút cho từng nhà cung cấp OAuth2 đã cấu hình. Bạn chỉ cần đảm bảo rằng các endpoint liên quan đến OAuth2 được bảo vệ đúng cách.

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class OAuth2LoginSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/", "/error", "/webjars/**").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> oauth2
                .loginPage("/oauth2/authorization/google") // Tùy chỉnh trang
                đăng nhập nếu cần
                .defaultSuccessUrl("/dashboard")
            )
        ;
    }
}
```

```

        )
        .logout(logout -> logout
            .logoutSuccessUrl("/").permitAll()
        );
    return http.build();
}
}

```

Sau khi đăng nhập thành công, bạn có thể truy cập thông tin người dùng từ `Authentication` object:

Java

```

import org.springframework.security.core.annotation.AuthenticationPrincipal;
import org.springframework.security.oauth2.core.user.OAuth2User;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class UserController {

    @GetMapping("/user/me")
    public Map<String, Object> user(@AuthenticationPrincipal OAuth2User
    oauth2User) {
        return oauth2User.getAttributes();
    }
}

```

6.6.2. OAuth2 Resource Server (Bảo vệ API)

Nếu ứng dụng của bạn cung cấp các RESTful API và bạn muốn bảo vệ chúng bằng OAuth2 (ví dụ: chấp nhận Access Token từ một Authorization Server), bạn cần cấu hình ứng dụng của mình như một OAuth2 Resource Server.

Bước 1: Thêm Dependency

XML

```

<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>

```

```
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>
```

Bước 2: Cấu hình trong `application.yml`

YAML

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: http://localhost:9000 # URL của Authorization Server
          # jwk-set-uri: http://localhost:9000/oauth2/jwks # Tùy chọn nếu
          issuer-uri không đủ
```

Spring Security sẽ tự động cấu hình để xác thực các JWT (JSON Web Token) được gửi trong header `Authorization: Bearer <token>`. Nó sẽ lấy khóa công khai từ `issuer-uri` (hoặc `jwk-set-uri`) để xác minh chữ ký của JWT.

Bước 3: Cấu hình `SecurityFilterChain`

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class OAuth2ResourceServerSecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/public/**").permitAll()
```

```

        .anyRequest().authenticated()
    )
    .oauth2ResourceServer(oauth2 -> oauth2.jwt(jwt -> {})); // Kích
hoạt OAuth2 Resource Server với JWT
    return http.build();
}
}

```

Với cấu hình này, bất kỳ request nào đến các endpoint được bảo vệ (ví dụ: `/api/data`) phải có một JWT hợp lệ trong header `Authorization`. Spring Security sẽ tự động xác thực JWT và tạo ra một `Authentication` object chứa thông tin từ JWT (ví dụ: các quyền, username).

Bạn có thể sử dụng các annotation như `@PreAuthorize` để kiểm soát quyền truy cập dựa trên các quyền (scopes) hoặc vai trò (roles) trong JWT:

Java

```

import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class ResourceController {

    @GetMapping("/api/data")
    @PreAuthorize("hasAuthority(\"'SCOPE_message.read'\")") // Yêu cầu scope
'message.read'
    public String getData() {
        return "Protected data!";
    }

    @GetMapping("/api/admin")
    @PreAuthorize("hasRole(\"'ADMIN'\")") // Yêu cầu vai trò 'ADMIN'
    public String getAdminData() {
        return "Admin data!";
    }
}

```

6.6.3. Lợi ích của OAuth2/OpenID Connect

- **Phân tách mối quan tâm:** Tách biệt xác thực và ủy quyền khỏi logic nghiệp vụ chính của ứng dụng.

- **Bảo mật:** Sử dụng các tiêu chuẩn công nghiệp đã được kiểm chứng.
- **Linh hoạt:** Hỗ trợ nhiều loại cấp quyền (grant types) và nhà cung cấp.
- **Single Sign-On (SSO):** Cho phép người dùng đăng nhập một lần và truy cập nhiều ứng dụng.
- **Microservices:** Lý tưởng để bảo vệ các microservices, nơi mỗi dịch vụ có thể hoạt động như một Resource Server.

6.1.2. Core Architecture (Kiến trúc cốt lõi) của Spring Security

Kiến trúc của Spring Security là một trong những điểm mạnh chính của nó, cho phép khả năng tùy biến cao và tích hợp sâu rộng. Nó được xây dựng dựa trên một chuỗi các bộ lọc Servlet (Servlet Filters) và các thành phần cốt lõi.

6.1.2.1. Servlet Filters

Spring Security hoạt động bằng cách chèn một chuỗi các bộ lọc Servlet vào chuỗi bộ lọc của ứng dụng web. Khi một request HTTP đến, nó sẽ đi qua từng bộ lọc trong chuỗi. Mỗi bộ lọc có một nhiệm vụ cụ thể liên quan đến bảo mật.

Bộ lọc quan trọng nhất là `FilterChainProxy`, nó là bộ lọc duy nhất được đăng ký trong `web.xml` (hoặc thông qua cấu hình Java) và chịu trách nhiệm ủy quyền cho một chuỗi các bộ lọc bảo mật khác của Spring Security. Điều này cho phép Spring Security có thể tùy chỉnh chuỗi bộ lọc bảo mật mà không cần thay đổi cấu hình Servlet Container.

Các bộ lọc chính trong chuỗi Spring Security (thứ tự có thể thay đổi tùy cấu hình):

- **`SecurityContextPersistenceFilter`** : Tải `SecurityContext` từ `HttpSession` (hoặc các nguồn khác) vào `SecurityContextHolder` ở đầu mỗi request và lưu lại vào `HttpSession` ở cuối request. `SecurityContextHolder` là nơi Spring Security lưu trữ thông tin về người dùng đã được xác thực.
- **`LogoutFilter`** : Xử lý các yêu cầu đăng xuất.

- **UsernamePasswordAuthenticationFilter** : Xử lý việc xác thực người dùng dựa trên username và password từ form đăng nhập.
- **BasicAuthenticationFilter** : Xử lý xác thực HTTP Basic.
- **FilterSecurityInterceptor** : Đây là bộ lọc cuối cùng trong chuỗi, chịu trách nhiệm ủy quyền (authorization). Nó quyết định xem người dùng có quyền truy cập vào tài nguyên được yêu cầu hay không dựa trên các quy tắc bảo mật đã cấu hình.
- **ExceptionTranslationFilter** : Chịu trách nhiệm dịch các `AuthenticationException` và `AccessDeniedException` thành các phản hồi HTTP phù hợp (ví dụ: chuyển hướng đến trang đăng nhập, trả về 403 Forbidden).

6.1.2.2. Các thành phần cốt lõi

Ngoài các bộ lọc, Spring Security còn có các thành phần cốt lõi khác:

- **SecurityContextHolder** : Một cơ chế để lưu trữ `SecurityContext` của người dùng hiện tại. Mặc định, nó sử dụng `ThreadLocal` để đảm bảo rằng `SecurityContext` có sẵn cho bất kỳ phương thức nào trong cùng một luồng xử lý request.
- **SecurityContext** : Chứa `Authentication` object của người dùng hiện tại.
- **Authentication** : Một interface đại diện cho thông tin xác thực của người dùng. Nó chứa `Principal` (đối tượng người dùng), `Credentials` (mật khẩu), và `Authorities` (các quyền/vai trò).
- **AuthenticationManager** : Một interface chính để thực hiện xác thực. Nó nhận một `Authentication` object (thường là `UsernamePasswordAuthenticationToken`) và trả về một `Authentication` object đã được xác thực. `ProviderManager` là một triển khai phổ biến của `AuthenticationManager`, nó ủy quyền việc xác thực cho một danh sách các `AuthenticationProvider`.
- **AuthenticationProvider** : Thực hiện logic xác thực cụ thể (ví dụ: `DaoAuthenticationProvider` cho xác thực dựa trên database, `JwtAuthenticationProvider` cho JWT).
- **UserDetailsService** : Một interface được sử dụng bởi `DaoAuthenticationProvider` để tải thông tin người dùng (username, password, roles) từ một nguồn dữ liệu (ví dụ:

database, LDAP) và chuyển đổi nó thành `UserDetails` object.

- **UserDetails** : Một interface đại diện cho thông tin chi tiết về người dùng (username, password, enabled, account non-expired, credentials non-expired, account non-locked, authorities).
- **AccessDecisionManager** : Một interface được sử dụng bởi `FilterSecurityInterceptor` để đưa ra quyết định ủy quyền. Nó nhận `Authentication` object, tài nguyên được bảo vệ, và các thuộc tính bảo mật (ví dụ: `hasRole('ADMIN')`) và quyết định xem người dùng có quyền truy cập hay không.
- **AccessDecisionVoter** : `AccessDecisionManager` thường sử dụng một hoặc nhiều `AccessDecisionVoter` để bỏ phiếu cho quyết định ủy quyền. Ví dụ: `RoleVoter` bỏ phiếu dựa trên vai trò của người dùng.

Kiến trúc này cho phép bạn thay thế hoặc mở rộng bất kỳ thành phần nào để phù hợp với yêu cầu bảo mật cụ thể của ứng dụng.

6.1.3. Authorization Filter (Bộ lọc ủy quyền)

Trong kiến trúc của Spring Security, khái niệm "Authorization Filter" thường được đề cập đến `FilterSecurityInterceptor` . Mặc dù Spring Security có một chuỗi các bộ lọc (filter chain) nơi mỗi bộ lọc có thể đóng góp vào quá trình bảo mật, `FilterSecurityInterceptor` là bộ lọc chính chịu trách nhiệm thực hiện quyết định ủy quyền cuối cùng.

6.1.3.1. Vai trò của `FilterSecurityInterceptor`

`FilterSecurityInterceptor` là bộ lọc cuối cùng trong chuỗi bộ lọc bảo mật của Spring Security. Nhiệm vụ chính của nó là:

- **Xác định tài nguyên được bảo vệ:** Nó xác định tài nguyên mà người dùng đang cố gắng truy cập (ví dụ: một URL, một phương thức).
- **Thu thập các thuộc tính bảo mật:** Nó lấy các thuộc tính bảo mật liên quan đến tài nguyên đó (ví dụ: `hasRole('ADMIN')` , `permitAll()` , `isAuthenticated()`).

- **Ủy quyền (Authorization):** Nó sử dụng `AccessDecisionManager` để quyết định xem người dùng hiện tại (được đại diện bởi `Authentication` object trong `SecurityContextHolder`) có quyền truy cập vào tài nguyên đó hay không, dựa trên các thuộc tính bảo mật đã thu thập.

6.1.3.2. Luồng hoạt động của `FilterSecurityInterceptor`

1. **Request đến:** Một request HTTP đến ứng dụng và đi qua các bộ lọc Spring Security trước đó (xác thực, quản lý session, v.v.).
2. **`SecurityContextHolder` đã có `Authentication` :** Tại thời điểm request đến `FilterSecurityInterceptor` , người dùng đã được xác thực và thông tin `Authentication` của họ đã được đặt trong `SecurityContextHolder` .
3. **Xác định tài nguyên và thuộc tính bảo mật:** `FilterSecurityInterceptor` xác định tài nguyên được yêu cầu (ví dụ: `/admin/users`) và tìm kiếm các cấu hình bảo mật liên quan đến tài nguyên đó (ví dụ: `http.authorizeHttpRequests().requestMatchers("/admin/**").hasRole("ADMIN")`). Các cấu hình này được chuyển đổi thành các `ConfigAttribute` .
4. **Gọi `AccessDecisionManager` :** `FilterSecurityInterceptor` sau đó ủy quyền quyết định ủy quyền cho `AccessDecisionManager` . Nó truyền vào `Authentication` object, tài nguyên được bảo vệ, và các `ConfigAttribute` .
5. **`AccessDecisionManager` đưa ra quyết định:** `AccessDecisionManager` (thường là `AffirmativeBased` , `ConsensusBased` , hoặc `UnanimousBased`) sẽ sử dụng một hoặc nhiều `AccessDecisionVoter` để bỏ phiếu. Ví dụ, `RoleVoter` sẽ kiểm tra xem `Authentication` object có vai trò (role) cần thiết hay không.
6. **Truy cập được cấp hoặc bị từ chối:**
 - Nếu `AccessDecisionManager` quyết định rằng người dùng có quyền truy cập, `FilterSecurityInterceptor` sẽ cho phép request tiếp tục đến Controller.
 - Nếu `AccessDecisionManager` từ chối quyền truy cập, `FilterSecurityInterceptor` sẽ ném ra một `AccessDeniedException` . Ngoại lệ này sau đó sẽ được `ExceptionHandler` xử

lý, thường dẫn đến việc trả về mã lỗi 403 Forbidden hoặc chuyển hướng đến trang lỗi truy cập bị từ chối.

6.1.3.3. Phân biệt với Authentication Filter

Điều quan trọng là phải phân biệt "Authorization Filter" (`FilterSecurityInterceptor`) với các "Authentication Filter" (như `UsernamePasswordAuthenticationFilter` , `BasicAuthenticationFilter` , `JwtAuthenticationFilter`).

- **Authentication Filters:** Nhiệm vụ chính là **xác minh danh tính** của người dùng (bạn là ai?) và tạo ra một `Authentication` object đã được xác thực.
- **Authorization Filter (`FilterSecurityInterceptor`):** Nhiệm vụ chính là **quyết định quyền truy cập** của người dùng đã được xác thực vào một tài nguyên cụ thể (bạn được phép làm gì?). Nó hoạt động dựa trên kết quả của quá trình xác thực.

Trong một ứng dụng Spring Security, quá trình xác thực luôn diễn ra trước quá trình ủy quyền. Người dùng phải được xác thực thành công trước khi hệ thống có thể quyết định xem họ có quyền truy cập vào tài nguyên được yêu cầu hay không.

6.3. Cấu hình Spring Security

Cấu hình Spring Security là quá trình định nghĩa các quy tắc và hành vi bảo mật cho ứng dụng của bạn. Trong Spring Boot, việc này chủ yếu được thực hiện thông qua các lớp cấu hình Java (Java-based configuration) sử dụng `@Configuration` và `@EnableWebSecurity` .

6.3.1. `SecurityFilterChain`

Kể từ Spring Security 5.7, `WebSecurityConfigurerAdapter` đã bị deprecated. Cách tiếp cận hiện đại để cấu hình Spring Security là định nghĩa một hoặc nhiều `SecurityFilterChain` beans. Mỗi `SecurityFilterChain` định nghĩa một chuỗi các bộ lọc bảo mật (security filters) sẽ được áp dụng cho các request HTTP cụ thể.

Một `SecurityFilterChain` được tạo ra bằng cách cấu hình một đối tượng `HttpSecurity` . Bạn có thể có nhiều `SecurityFilterChain` beans, và Spring Security sẽ áp dụng `SecurityFilterChain` đầu tiên khớp với request.

Ví dụ cấu hình cơ bản:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authz -> authz
                .requestMatchers("/public/**").permitAll() // Cho phép truy
cập công khai
                .requestMatchers("/admin/**").hasRole("ADMIN") // Yêu cầu vai
trò ADMIN
                .anyRequest().authenticated() // Tất cả các request khác yêu
cầu xác thực
            )
            .formLogin(form -> form
                .loginPage("/login") // Trang đăng nhập tùy chỉnh
                .defaultSuccessUrl("/dashboard") // Chuyển hướng sau khi đăng
nhập thành công
                .permitAll() // Cho phép truy cập trang đăng nhập
            )
            .logout(logout -> logout
                .logoutUrl("/logout") // URL đăng xuất
                .logoutSuccessUrl("/login?logout") // Chuyển hướng sau khi
đăng xuất
                .permitAll()
            )
            .csrf(csrf -> csrf.disable()); // Tắt CSRF cho API (cần cân nhắc
kỹ)

        return http.build();
    }
}
```

Trong ví dụ trên:

- `@Configuration` : Đánh dấu lớp này là một lớp cấu hình Spring.
- `@EnableWebSecurity` : Kích hoạt tính năng bảo mật web của Spring Security.
- `securityFilterChain(HttpSecurity http)` : Phương thức này định nghĩa một `SecurityFilterChain` bằng cách cấu hình đối tượng `HttpSecurity` .

6.3.2. `HttpSecurity` và các phương thức cấu hình

Đối tượng `HttpSecurity` là trung tâm của cấu hình bảo mật, cho phép bạn định nghĩa các quy tắc bảo mật cho các request HTTP. Nó cung cấp một API fluent để cấu hình nhiều khía cạnh khác nhau của bảo mật:

- `authorizeHttpRequests()` : Bắt đầu cấu hình các quy tắc ủy quyền dựa trên request HTTP.
 - `requestMatchers(String... patterns)` : Chỉ định các URL pattern để áp dụng quy tắc.
 - `permitAll()` : Cho phép tất cả mọi người truy cập mà không cần xác thực.
 - `authenticated()` : Yêu cầu người dùng phải được xác thực.
 - `hasRole(String role)` : Yêu cầu người dùng phải có một vai trò cụ thể (ví dụ: `hasRole("ADMIN")` sẽ kiểm tra `ROLE_ADMIN`).
 - `hasAnyRole(String... roles)` : Yêu cầu người dùng phải có ít nhất một trong các vai trò được chỉ định.
 - `hasAuthority(String authority)` : Yêu cầu người dùng phải có một quyền cụ thể (ví dụ: `hasAuthority("READ_PRIVILEGE")`).
 - `hasAnyAuthority(String... authorities)` : Yêu cầu người dùng phải có ít nhất một trong các quyền được chỉ định.
 - `access(String expression)` : Cho phép sử dụng Spring Expression Language (SpEL) để định nghĩa các quy tắc phức tạp hơn.
 - `anyRequest()` : Áp dụng quy tắc cho bất kỳ request nào không khớp với các `requestMatchers` trước đó.

- **formLogin()** : Cấu hình xác thực dựa trên form đăng nhập.
 - `loginPage(String path)` : Chỉ định URL của trang đăng nhập tùy chỉnh.
 - `defaultSuccessUrl(String url)` : URL chuyển hướng sau khi đăng nhập thành công.
 - `failureUrl(String url)` : URL chuyển hướng khi đăng nhập thất bại.
 - `usernameParameter(String param)` : Tên tham số cho username trong form.
 - `passwordParameter(String param)` : Tên tham số cho password trong form.
- **httpBasic()** : Cấu hình xác thực HTTP Basic.
- **logout()** : Cấu hình chức năng đăng xuất.
 - `logoutUrl(String url)` : URL để kích hoạt đăng xuất.
 - `logoutSuccessUrl(String url)` : URL chuyển hướng sau khi đăng xuất thành công.
 - `invalidateHttpSession(boolean invalidate)` : Vô hiệu hóa session HTTP sau khi đăng xuất.
 - `deleteCookies(String... cookieNames)` : Xóa các cookie sau khi đăng xuất.
- **csrf()** : Cấu hình bảo vệ CSRF (Cross-Site Request Forgery).
 - `csrf().disable()` : Tắt bảo vệ CSRF (thường được sử dụng cho các RESTful API không sử dụng session).
 - `csrf().ignoringRequestMatchers(String... patterns)` : Bỏ qua bảo vệ CSRF cho các URL cụ thể.
- **sessionManagement()** : Cấu hình quản lý session.
 - `maximumSessions(int maxSessions)` : Giới hạn số lượng session đồng thời cho một người dùng.
 - `sessionCreationPolicy(SessionCreationPolicy policy)` : Định nghĩa cách session được tạo (ví dụ: `STATELESS` cho REST APIs).
- **exceptionHandling()** : Cấu hình xử lý ngoại lệ bảo mật.

- `authenticationEntryPoint(AuthenticationEntryPoint entryPoint)` : Xử lý khi người dùng chưa được xác thực cố gắng truy cập tài nguyên được bảo vệ.
- `accessDeniedHandler(AccessDeniedHandler handler)` : Xử lý khi người dùng đã xác thực nhưng không có quyền truy cập tài nguyên.

6.3.3. Cấu hình `WebSecurity` (Ignoring Security)

Trong một số trường hợp, bạn có thể muốn bỏ qua hoàn toàn Spring Security cho một số request nhất định, ví dụ như các tài nguyên tĩnh (CSS, JavaScript, hình ảnh) hoặc các endpoint công khai không cần bảo mật. Bạn có thể làm điều này bằng cách cấu hình `WebSecurity` .

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.WebSecurity;
import
org.springframework.security.config.annotation.web.configuration.WebSecurityC
ustomizer;

@Configuration
public class WebSecurityConfig {

    @Bean
    public WebSecurityCustomizer webSecurityCustomizer() {
        return (web) -> web.ignoring().requestMatchers("/resources/**",
"/static/**", "/css/**", "/js/**", "/images/**", "/webjars/**");
    }
}
```

Sử dụng `WebSecurityCustomizer` là cách hiện đại để cấu hình `WebSecurity` . Các request khớp với các pattern trong `ignoring()` sẽ không đi qua chuỗi bộ lọc bảo mật của Spring Security, giúp cải thiện hiệu suất cho các tài nguyên không cần bảo vệ.

6.3.4. Cấu hình Password Encoder

Spring Security yêu cầu một `PasswordEncoder` để mã hóa mật khẩu. `BCryptPasswordEncoder` là một lựa chọn phổ biến và được khuyến nghị.

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;

@Configuration
public class PasswordEncoderConfig {

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder();
    }
}
```

Bằng cách sử dụng các phương thức của `HttpSecurity` và `WebSecurity`, bạn có thể xây dựng một cấu hình bảo mật mạnh mẽ và tùy chỉnh cho ứng dụng Spring Boot của mình.

6.4. User Authentication (Xác thực người dùng)

Xác thực là quá trình xác minh danh tính của người dùng. Trong Spring Security, quá trình này được điều phối bởi `AuthenticationManager`.

6.4.1. `AuthenticationManager` và `AuthenticationProvider`

- **`AuthenticationManager`** : Đây là interface chính để thực hiện xác thực. Phương thức `authenticate(Authentication authentication)` của nó nhận một đối tượng `Authentication` (thường là `UsernamePasswordAuthenticationToken` chứa thông tin đăng nhập của người dùng) và trả về một đối tượng `Authentication` đã được xác thực nếu thành công, hoặc ném ra `AuthenticationException` nếu thất bại.
- **`ProviderManager`** : Là một triển khai phổ biến của `AuthenticationManager`. Nó không tự thực hiện xác thực mà ủy quyền cho một danh sách các `AuthenticationProvider`.
- **`AuthenticationProvider`** : Là nơi chứa logic xác thực thực tế. Mỗi `AuthenticationProvider` có thể xử lý một loại xác thực cụ thể (ví dụ: xác thực bằng username/password, xác thực bằng JWT, xác thực OAuth2). Khi `ProviderManager` nhận một yêu cầu xác thực, nó sẽ duyệt qua danh sách các `AuthenticationProvider` của mình và hỏi từng provider xem nó

có hỗ trợ loại `Authentication` được yêu cầu hay không. Nếu có, provider đó sẽ thực hiện xác thực.

6.4.2. `UserDetailsService` và `UserDetails`

- **`UserDetailsService`** : Đây là một interface quan trọng được sử dụng bởi các `AuthenticationProvider` (như `DaoAuthenticationProvider` cho xác thực dựa trên database) để tải thông tin chi tiết về người dùng. Phương thức `loadUserByUsername(String username)` của nó sẽ tìm kiếm người dùng theo username và trả về một đối tượng `UserDetails` .
- **`UserDetails`** : Là một interface đại diện cho thông tin chi tiết về người dùng (username, password, các quyền/vai trò, trạng thái tài khoản như có bị khóa hay không, có hết hạn hay không). Spring Security sử dụng đối tượng này để lưu trữ thông tin người dùng đã được xác thực trong `SecurityContext` .

6.4.3. Các cơ chế xác thực phổ biến

Spring Security hỗ trợ nhiều cơ chế xác thực khác nhau:

- **Form-based Authentication**: Người dùng cung cấp username và password thông qua một form HTML. Spring Security sẽ tự động xử lý việc gửi form, xác thực và chuyển hướng.
 - Cấu hình: `http.formLogin()`
 - Thường sử dụng `UsernamePasswordAuthenticationFilter` .
- **HTTP Basic Authentication**: Username và password được gửi trong header `Authorization` của request HTTP, được mã hóa Base64. Thường được dùng cho các API đơn giản hoặc cho các client không có giao diện người dùng.
 - Cấu hình: `http.httpBasic()`
 - Thường sử dụng `BasicAuthenticationFilter` .
- **JWT (JSON Web Token) Authentication**: Người dùng đăng nhập một lần để nhận một JWT. Các request tiếp theo sẽ gửi JWT trong header `Authorization` . JWT là stateless, phù hợp cho RESTful APIs và microservices.

- Yêu cầu triển khai `OncePerRequestFilter` tùy chỉnh để trích xuất và xác thực JWT.
- Cấu hình: Thêm filter tùy chỉnh vào chuỗi bảo mật (`http.addFilterBefore(...)`).
- **OAuth2/OpenID Connect Authentication:** Sử dụng một Authorization Server bên ngoài để xác thực người dùng và cấp phát token. Ứng dụng của bạn đóng vai trò là OAuth2 Client hoặc Resource Server.
- Cấu hình: `spring-boot-starter-oauth2-client` hoặc `spring-boot-starter-oauth2-resource-server` .

6.4.4. Luồng xác thực cơ bản

1. Người dùng gửi thông tin đăng nhập (ví dụ: username/password) đến ứng dụng.
2. Spring Security tạo một đối tượng `Authentication` chưa được xác thực (ví dụ: `UsernamePasswordAuthenticationToken`).
3. Đối tượng `Authentication` này được gửi đến `AuthenticationManager` .
4. `AuthenticationManager` ủy quyền cho các `AuthenticationProvider` phù hợp.
5. `AuthenticationProvider` (ví dụ: `DaoAuthenticationProvider`) sử dụng `UserDetailsService` để tải `UserDetails` từ nguồn dữ liệu.
6. `AuthenticationProvider` so sánh mật khẩu được cung cấp với mật khẩu đã mã hóa trong `UserDetails` .
7. Nếu khớp, `AuthenticationProvider` trả về một đối tượng `Authentication` đã được xác thực (chứa `UserDetails` và các quyền/vai trò).
8. Đối tượng `Authentication` đã xác thực này được lưu trữ trong `SecurityContextHolder` , làm cho thông tin người dùng có sẵn cho các request tiếp theo trong cùng một luồng.

6.5. Authorization (Ủy quyền)

Ủy quyền là quá trình xác định xem một người dùng đã được xác thực có quyền truy cập vào một tài nguyên cụ thể hoặc thực hiện một hành động cụ thể hay không. Trong Spring

Security, ủy quyền được thực hiện sau khi xác thực thành công.

6.5.1. Các cấp độ ủy quyền

Spring Security cung cấp ủy quyền ở nhiều cấp độ:

- **Web-level Security (Ủy quyền cấp độ Web):** Kiểm soát quyền truy cập vào các URL hoặc HTTP method. Đây là cách phổ biến nhất để bảo vệ các endpoint trong ứng dụng web và RESTful API.
 - Được cấu hình thông qua `HttpSecurity.authorizeHttpRequests()` trong `SecurityFilterChain`.
 - Sử dụng các biểu thức như `permitAll()`, `authenticated()`, `hasRole()`, `hasAuthority()`, `hasAnyRole()`, `hasAnyAuthority()`.
 - Ví dụ: `http.authorizeHttpRequests(authz -> authz.requestMatchers("/admin/**").hasRole("ADMIN").anyRequest().authenticated())`
- **Method-level Security (Ủy quyền cấp độ Phương thức):** Kiểm soát quyền truy cập vào các phương thức cụ thể trong các lớp dịch vụ (Service) hoặc Repository. Điều này cho phép kiểm soát chi tiết hơn về quyền truy cập vào logic nghiệp vụ.
 - Sử dụng các annotation như `@PreAuthorize`, `@PostAuthorize`, `@PreFilter`, `@PostFilter`.
 - Yêu cầu kích hoạt bằng `@EnableMethodSecurity` (hoặc `@EnableGlobalMethodSecurity` cũ hơn).
- **Domain Object Security (Ủy quyền đối tượng miền):** Kiểm soát quyền truy cập vào các đối tượng dữ liệu cụ thể (ví dụ: người dùng chỉ có thể chỉnh sửa bài viết của chính họ). Đây là cấp độ ủy quyền chi tiết nhất và thường yêu cầu triển khai tùy chỉnh hoặc sử dụng các thư viện như Spring Security ACL (Access Control List).

6.5.2. Các biểu thức ủy quyền phổ biến

Spring Security sử dụng Spring Expression Language (SpEL) để định nghĩa các quy tắc ủy quyền. Một số biểu thức phổ biến bao gồm:

- **permitAll()** : Cho phép truy cập mà không cần xác thực.
- **denyAll()** : Từ chối tất cả các truy cập.
- **authenticated()** : Yêu cầu người dùng phải được xác thực.
- **fullyAuthenticated()** : Yêu cầu người dùng phải được xác thực đầy đủ (không phải remember-me).
- **hasRole('ROLE_NAME')** : Yêu cầu người dùng có vai trò cụ thể. Spring Security tự động thêm tiền tố **ROLE_** nếu bạn sử dụng **hasRole()** .
- **hasAnyRole('ROLE_NAME1', 'ROLE_NAME2')** : Yêu cầu người dùng có ít nhất một trong các vai trò được chỉ định.
- **hasAuthority('AUTHORITY_NAME')** : Yêu cầu người dùng có một quyền cụ thể. Quyền này có thể là bất kỳ chuỗi nào (ví dụ: **READ_PRIVILEGE** , **DELETE_PRODUCT**).
- **hasAnyAuthority('AUTHORITY_NAME1', 'AUTHORITY_NAME2')** : Yêu cầu người dùng có ít nhất một trong các quyền được chỉ định.
- **isAnonymous()** : Người dùng chưa được xác thực.
- **isRememberMe()** : Người dùng được xác thực thông qua remember-me.
- **isFullyAuthenticated()** : Người dùng được xác thực đầy đủ (không phải remember-me hoặc anonymous).
- **principal** : Tham chiếu đến đối tượng **Principal** của người dùng hiện tại (thường là **UserDetails**).
- **authentication** : Tham chiếu đến đối tượng **Authentication** của người dùng hiện tại.

6.5.3. Vai trò (Roles) và Quyền (Authorities)

Trong Spring Security:

- **Authority (Quyền):** Là một quyền hạn cụ thể mà người dùng có. Ví dụ: **READ_PRODUCT** , **WRITE_PRODUCT** , **DELETE_PRODUCT** . Đây là khái niệm chi tiết hơn.

- **Role (Vai trò):** Là một tập hợp các quyền hạn. Ví dụ: vai trò `ADMIN` có thể bao gồm các quyền `READ_PRODUCT`, `WRITE_PRODUCT`, `DELETE_PRODUCT`, `MANAGE_USERS`. Vai trò thường được tiền tố bằng `ROLE_` theo quy ước của Spring Security (ví dụ: `ROLE_ADMIN`).

Bạn có thể sử dụng cả hai khái niệm này để tổ chức quyền truy cập trong ứng dụng của mình. Đối với các kiểm tra ủy quyền, `hasRole()` là một cách viết tắt tiện lợi cho `hasAuthority('ROLE_')`.

6.5.4. `AccessDecisionManager` và `AccessDecisionVoter`

Như đã đề cập trong phần kiến trúc, `AccessDecisionManager` là thành phần chịu trách nhiệm đưa ra quyết định ủy quyền cuối cùng. Nó sử dụng một hoặc nhiều `AccessDecisionVoter` để bỏ phiếu về việc liệu người dùng có được phép truy cập tài nguyên hay không. Các `AccessDecisionVoter` phổ biến bao gồm:

- **`RoleVoter`** : Bỏ phiếu dựa trên các vai trò của người dùng.
- **`AuthenticatedVoter`** : Bỏ phiếu dựa trên việc người dùng đã được xác thực hay chưa.
- **`WebExpressionVoter`** : Xử lý các biểu thức SpEL được sử dụng trong `authorizeHttpRequests()` và `@PreAuthorize`.

Cơ chế bỏ phiếu này cho phép bạn tùy chỉnh hành vi ủy quyền một cách linh hoạt, ví dụ: yêu cầu tất cả các voter phải đồng ý (`UnanimousBased`), hoặc chỉ cần một voter đồng ý (`AffirmativeBased`).

6.6. Method Security (Bảo mật cấp độ phương thức)

Ngoài việc bảo vệ các URL (Web-level Security), Spring Security còn cho phép bạn bảo vệ các phương thức cụ thể trong các lớp dịch vụ (Service) hoặc Repository. Điều này cung cấp một lớp bảo mật chi tiết hơn, đảm bảo rằng chỉ những người dùng có quyền phù hợp mới có thể gọi các phương thức nhất định.

6.6.1. Kích hoạt Method Security

Để sử dụng Method Security, bạn cần kích hoạt nó trong cấu hình Spring Security của mình. Kể từ Spring Security 5.6, bạn có thể sử dụng `@EnableMethodSecurity` .

Java

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity;

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true,
jsr250Enabled = true)
public class MethodSecurityConfig {
    // Cấu hình khác nếu cần
}
```

- `prePostEnabled = true` : Kích hoạt các annotation `@PreAuthorize` và `@PostAuthorize` .
- `securedEnabled = true` : Kích hoạt annotation `@Secured` .
- `jsr250Enabled = true` : Kích hoạt các annotation chuẩn JSR-250 như `@RolesAllowed` .

6.6.2. Các Annotation phổ biến

6.6.2.1. `@PreAuthorize`

`@PreAuthorize` được sử dụng để kiểm tra quyền truy cập **trước khi** phương thức được thực thi. Nếu biểu thức SpEL trả về `false` , phương thức sẽ không được gọi và một `AccessDeniedException` sẽ được ném ra.

Ví dụ:

Java

```
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    @PreAuthorize("hasRole(\"ADMIN\")")
    public void deleteProduct(Long productId) {
```

```

        // Chỉ người dùng có vai trò ADMIN mới có thể xóa sản phẩm
        System.out.println("Deleting product with ID: " + productId);
    }

    @PreAuthorize("hasAnyRole(\ 'ADMIN\ ', \ 'MANAGER\ ')")
    public Product updateProduct(Product product) {
        // Chỉ ADMIN hoặc MANAGER mới có thể cập nhật sản phẩm
        return productRepository.save(product);
    }

    @PreAuthorize("hasPermission(#productId, \ 'Product\ ', \ 'read\ ')")
    public Product getProductDetails(Long productId) {
        // Kiểm tra quyền đọc trên đối tượng Product cụ thể
        return productRepository.findById(productId).orElse(null);
    }

    @PreAuthorize("#username == authentication.principal.username")
    public UserProfile getUserProfile(String username) {
        // Chỉ người dùng có username khớp với username trong tham số mới có
        // thể xem profile của chính họ
        return userRepository.findByUsername(username).orElse(null);
    }
}

```

6.6.2.2. @PostAuthorize

`@PostAuthorize` được sử dụng để kiểm tra quyền truy cập **sau khi** phương thức đã được thực thi nhưng **trước khi** kết quả được trả về cho người gọi. Điều này hữu ích khi quyết định ủy quyền phụ thuộc vào giá trị trả về của phương thức.

Ví dụ:

Java

```

import org.springframework.security.access.prepost.PostAuthorize;
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @PostAuthorize("returnObject.customer.username ==
authentication.principal.username or hasRole(\ 'ADMIN\ ')")
    public Order getOrderById(Long orderId) {
        Order order = orderRepository.findById(orderId).orElse(null);
        // Chỉ cho phép khách hàng xem đơn hàng của chính họ hoặc ADMIN
    }
}

```

```
        return order;
    }
}
```

6.6.2.3. @PreFilter

`@PreFilter` được sử dụng để lọc các phần tử của một Collection hoặc Array được truyền vào làm tham số của phương thức **trước khi** phương thức được thực thi. Biểu thức SpEL được đánh giá cho từng phần tử.

Ví dụ:

Java

```
import org.springframework.security.access.prepost.PreFilter;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

    @PreFilter("filterObject.owner.username ==
authentication.principal.username or hasRole(\ 'ADMIN\ ')")
    public void saveProducts(List<Product> products) {
        // Chỉ lưu các sản phẩm mà người dùng hiện tại là chủ sở hữu hoặc nếu
        người dùng là ADMIN
        productRepository.saveAll(products);
    }
}
```

- `filterObject` : Tham chiếu đến phần tử hiện tại trong Collection/Array đang được lọc.

6.6.2.4. @PostFilter

`@PostFilter` được sử dụng để lọc các phần tử của một Collection hoặc Array được trả về bởi phương thức **sau khi** phương thức được thực thi. Biểu thức SpEL được đánh giá cho từng phần tử của Collection/Array trả về.

Ví dụ:

Java

```
import org.springframework.security.access.prepost.PostFilter;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class ProductService {

    @PostFilter("filterObject.available == true or hasRole(\"ADMIN\")")
    public List<Product> getAllProducts() {
        // Chỉ trả về các sản phẩm có sẵn hoặc tất cả sản phẩm nếu người dùng
        // là ADMIN
        return productRepository.findAll();
    }
}
```

- `filterObject` : Tham chiếu đến phần tử hiện tại trong Collection/Array trả về đang được lọc.

6.6.2.5. `@Secured` và `@RolesAllowed`

- **`@Secured`** : Annotation này là một phần của Spring Security và cho phép bạn chỉ định các vai trò cần thiết để truy cập một phương thức.
 - **Ví dụ:** `@Secured({"ROLE_ADMIN", "ROLE_USER"})`
- **`@RolesAllowed`** : Annotation này là một phần của chuẩn JSR-250 và có chức năng tương tự như `@Secured` .
 - **Ví dụ:** `@RolesAllowed({"ADMIN", "USER"})`

Cả hai annotation này đơn giản hơn `@PreAuthorize` và `@PostAuthorize` vì chúng chỉ hỗ trợ kiểm tra vai trò/quyền, không hỗ trợ các biểu thức SpEL phức tạp.

6.6.3. SpEL trong Method Security

Spring Expression Language (SpEL) là một ngôn ngữ biểu thức mạnh mẽ được sử dụng trong Spring Security để định nghĩa các quy tắc ủy quyền phức tạp. Nó cho phép bạn truy cập vào các đối tượng như `authentication` (thông tin người dùng hiện tại), các tham số của phương thức, và thậm chí cả giá trị trả về của phương thức.

Các đối tượng có sẵn trong SpEL context:

- `authentication` : Đối tượng `Authentication` hiện tại.
- `principal` : Đối tượng `Principal` từ `authentication` (thường là `UserDetails`).
- `hasRole("ROLE_NAME")` , `hasAnyRole(...)` , `hasAuthority("AUTHORITY_NAME")` , `hasAnyAuthority(...)` .
- `isAnonymous()` , `isAuthenticated()` , `isFullyAuthenticated()` , `isRememberMe()` .
- Các tham số của phương thức (ví dụ: `#productId` , `#username`).
- `returnObject` : Giá trị trả về của phương thức (chỉ dùng với `@PostAuthorize` và `@PostFilter`).

Method Security là một công cụ mạnh mẽ để áp dụng các quy tắc bảo mật chi tiết và linh hoạt trong ứng dụng Spring Boot của bạn, đặc biệt là khi bạn cần kiểm soát quyền truy cập dựa trên logic nghiệp vụ hoặc dữ liệu cụ thể.

6.1. Giới thiệu chuyên sâu về Spring Security

Spring Security là một framework bảo mật mạnh mẽ và có khả năng tùy biến cao, cung cấp các giải pháp xác thực (authentication) và ủy quyền (authorization) toàn diện cho các ứng dụng Java, đặc biệt là các ứng dụng được xây dựng trên nền tảng Spring Framework. Mục tiêu chính của Spring Security là bảo vệ ứng dụng khỏi các mối đe dọa bảo mật phổ biến bằng cách cung cấp một bộ công cụ và cơ chế tiêu chuẩn, giúp các nhà phát triển dễ dàng tích hợp các tính năng bảo mật mà không cần phải tự xây dựng từ đầu.

Khi tích hợp với Spring Boot, việc cấu hình Spring Security trở nên đơn giản hơn rất nhiều nhờ vào các cấu hình mặc định thông minh (auto-configuration) và các starter dependency. Spring Boot tự động nhận diện sự hiện diện của `spring-boot-starter-security` và áp dụng các thiết lập bảo mật cơ bản, giúp bạn nhanh chóng có một ứng dụng được bảo vệ.

6.1.1. Tại sao cần Spring Security?

Việc tự xây dựng một hệ thống bảo mật từ đầu là một nhiệm vụ phức tạp và dễ mắc lỗi. Các vấn đề như tấn công Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), SQL Injection, hoặc quản lý phiên (session management) không đúng cách có thể dẫn đến các lỗ

hỗ trợ bảo mật nghiêm trọng. Spring Security giải quyết những thách thức này bằng cách cung cấp:

- **Các tính năng bảo mật đã được kiểm chứng:** Framework đã được cộng đồng kiểm tra và sử dụng rộng rãi, giảm thiểu rủi ro từ các lỗi bảo mật phổ biến.
- **Khả năng tùy biến cao:** Mặc dù có cấu hình mặc định mạnh mẽ, Spring Security cho phép tùy chỉnh gần như mọi khía cạnh của quá trình xác thực và ủy quyền để phù hợp với các yêu cầu nghiệp vụ cụ thể.
- **Tích hợp sâu với Spring Ecosystem:** Hoạt động liền mạch với Spring MVC, Spring WebFlux, Spring Data, và các module Spring khác.
- **Hỗ trợ nhiều cơ chế xác thực:** Bao gồm form-based login, HTTP Basic/Digest authentication, OAuth2, OpenID Connect, LDAP, SAML, JWT, v.v.
- **Bảo vệ chống lại các cuộc tấn công phổ biến:** Tích hợp sẵn các biện pháp bảo vệ chống lại CSRF, session fixation, clickjacking, v.v.

6.1.2. Các khái niệm cốt lõi trong Spring Security

Để hiểu rõ cách Spring Security hoạt động, cần nắm vững các khái niệm sau:

- **Authentication (Xác thực):** Quá trình xác minh danh tính của người dùng. Đây là bước đầu tiên để xác định "bạn là ai?". Spring Security cung cấp một kiến trúc xác thực linh hoạt, cho phép bạn tích hợp với nhiều nguồn dữ liệu người dùng khác nhau (in-memory, database, LDAP, OAuth2).
- **Authentication** : Một interface đại diện cho thông tin xác thực của người dùng. Nó chứa **Principal** (đối tượng đại diện cho người dùng), **Credentials** (thông tin xác thực như mật khẩu), và **Authorities** (các quyền).
- **AuthenticationManager** : Interface chính để thực hiện xác thực. Nó nhận một đối tượng **Authentication** (thường là **UsernamePasswordAuthenticationToken**) và trả về một đối tượng **Authentication** đã được xác thực nếu thành công.
- **ProviderManager** : Một triển khai phổ biến của **AuthenticationManager** có thể ủy quyền cho một danh sách các **AuthenticationProvider** .

- **AuthenticationProvider** : Thực hiện logic xác thực cụ thể cho một loại thông tin xác thực nhất định (ví dụ: `DaoAuthenticationProvider` cho username/password từ database).
- **Authorization (Ủy quyền)**: Quá trình xác định xem người dùng đã được xác thực có quyền truy cập vào một tài nguyên cụ thể hay thực hiện một hành động cụ thể hay không. Đây là bước để trả lời câu hỏi "bạn được phép làm gì?".
- **AccessDecisionManager** : Interface chính để đưa ra quyết định ủy quyền. Nó nhận thông tin xác thực của người dùng, đối tượng được bảo vệ (ví dụ: URL, phương thức), và các thuộc tính cấu hình bảo mật (ví dụ: `ROLE_ADMIN`).
- **AccessDecisionVoter** : Một thành phần của `AccessDecisionManager` bỏ phiếu cho việc truy cập (GRANT, DENY, ABSTAIN).
- **GrantedAuthority** : Một interface đại diện cho một quyền cụ thể được cấp cho `Principal` . Thường được triển khai dưới dạng `SimpleGrantedAuthority` hoặc các đối tượng `Role` tùy chỉnh.
- **Principal**: Đại diện cho người dùng hiện tại đang tương tác với ứng dụng. Trong Spring Security, `Principal` thường là một đối tượng `UserDetails` .
- **UserDetails** : Một interface cốt lõi trong Spring Security, đại diện cho thông tin chi tiết về người dùng (username, password, enabled status, account expiration, credentials expiration, account locked status, và danh sách các `GrantedAuthority`).
- **UserDetailsService** : Một interface được sử dụng để tải thông tin chi tiết về người dùng (`UserDetails`) dựa trên username. Đây là cầu nối giữa Spring Security và nguồn dữ liệu người dùng của bạn.
- **Security Context**: Chứa thông tin bảo mật của người dùng hiện tại. `SecurityContextHolder` là một lớp tiện ích cung cấp quyền truy cập vào `SecurityContext` hiện tại. Thông tin này được lưu trữ theo từng thread và được duy trì trong suốt quá trình xử lý request.
- **Security Filter Chain**: Spring Security hoạt động dựa trên một chuỗi các bộ lọc Servlet (Servlet Filters). Mỗi bộ lọc có một trách nhiệm cụ thể (ví dụ: xác thực, ủy quyền, xử lý session). Các request HTTP đi qua chuỗi bộ lọc này trước khi đến được

DispatcherServlet của Spring MVC. Điều này cho phép Spring Security chặn và xử lý các request ở giai đoạn rất sớm.

Việc hiểu rõ các khái niệm này là nền tảng để bạn có thể cấu hình và tùy chỉnh Spring Security một cách hiệu quả cho các ứng dụng của mình.

6.3. Cấu hình Spring Security

Cấu hình Spring Security là trái tim của việc bảo vệ ứng dụng của bạn. Với Spring Boot, việc này được đơn giản hóa đáng kể thông qua auto-configuration và các DSL (Domain Specific Language) trong `HttpSecurity`. Lớp cấu hình chính thường kế thừa từ `WebSecurityConfigurerAdapter` (đã deprecated từ Spring Security 5.7) hoặc sử dụng phương pháp cấu hình dựa trên `SecurityFilterChain` Bean.

6.3.1. `SecurityFilterChain` Bean (Phương pháp hiện đại)

Từ Spring Security 5.7 trở đi, cách tiếp cận khuyến nghị để cấu hình bảo mật là khai báo một `SecurityFilterChain` bean. Điều này giúp loại bỏ sự cần thiết của việc kế thừa `WebSecurityConfigurerAdapter` và cho phép cấu hình linh hoạt hơn.

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity // Kích hoạt Spring Security's web security support
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws
Exception {
        http
            // Cấu hình ủy quyền cho các request HTTP
            .authorizeHttpRequests(authorize -> authorize
```

```

        // Cho phép truy cập công khai vào các đường dẫn này
        .requestMatchers("/public/**", "/api/auth/**", "/css/**",
"/js/**", "/images/**").permitAll()
        // Yêu cầu vai trò ADMIN để truy cập các đường dẫn /admin/**
        .requestMatchers("/admin/**").hasRole("ADMIN")
        // Yêu cầu vai trò USER hoặc ADMIN để truy cập các đường dẫn
/api/**

        .requestMatchers("/api/**").hasAnyRole("USER", "ADMIN")
        // Tất cả các request khác đều yêu cầu xác thực
        .anyRequest().authenticated()
    )
    // Cấu hình form login
    .formLogin(form -> form
        .loginPage("/login") // Chỉ định trang login tùy chỉnh
        .defaultSuccessUrl("/dashboard") // URL chuyển hướng sau khi
login thành công
        .permitAll() // Cho phép tất cả mọi người truy cập trang
login
    )
    // Cấu hình logout
    .logout(logout -> logout
        .logoutUrl("/logout") // URL để xử lý logout
        .logoutSuccessUrl("/login?logout") // URL chuyển hướng sau
khi logout thành công
        .permitAll() // Cho phép tất cả mọi người truy cập logout
    )
    // Cấu hình CSRF (Cross-Site Request Forgery) protection
    // Mặc định Spring Security bật CSRF. Tắt nó cho các API
stateless (ví dụ: REST API với JWT)
    .csrf(csrf -> csrf.disable());

    return http.build();
}

// Các Bean khác như UserDetailsService, PasswordEncoder sẽ được định
nghĩa ở đây
}

```

Giải thích các thành phần chính của `HttpSecurity` :

- `authorizeHttpRequests()` : Đây là điểm khởi đầu để cấu hình các quy tắc ủy quyền dựa trên request HTTP. Nó cho phép bạn định nghĩa những đường dẫn nào cần được bảo vệ và bằng cách nào.

- **requestMatchers()** : Chỉ định các mẫu đường dẫn (URL patterns) để áp dụng quy tắc. Có thể sử dụng Ant-style patterns (`/admin/**`) hoặc Regex patterns.
- **permitAll()** : Cho phép tất cả mọi người truy cập vào đường dẫn này mà không cần xác thực.
- **authenticated()** : Yêu cầu người dùng phải được xác thực để truy cập.
- **hasRole("ROLE_NAME")** : Yêu cầu người dùng phải có vai trò (role) cụ thể. Spring Security tự động thêm tiền tố `ROLE_` nếu bạn sử dụng `hasRole()` . Ví dụ: `hasRole("ADMIN")` sẽ kiểm tra xem người dùng có quyền `ROLE_ADMIN` hay không.
- **hasAnyRole("ROLE1", "ROLE2")** : Yêu cầu người dùng có ít nhất một trong các vai trò được liệt kê.
- **hasAuthority("PERMISSION")** : Yêu cầu người dùng có một quyền cụ thể (ví dụ: `hasAuthority("DELETE_PRODUCT")`).
- **hasAnyAuthority("PERMISSION1", "PERMISSION2")** : Yêu cầu người dùng có ít nhất một trong các quyền được liệt kê.
- **anyRequest()** : Áp dụng cho tất cả các request còn lại không khớp với các `requestMatchers` trước đó.
- **formLogin()** : Cấu hình xác thực dựa trên form HTML. Khi người dùng cố gắng truy cập một tài nguyên được bảo vệ mà chưa được xác thực, họ sẽ được chuyển hướng đến trang đăng nhập.
- **loginPage("/login")** : Chỉ định URL của trang đăng nhập tùy chỉnh của bạn. Nếu không cấu hình, Spring Security sẽ tạo một trang đăng nhập mặc định.
- **defaultSuccessUrl("/dashboard")** : URL mà người dùng sẽ được chuyển hướng đến sau khi đăng nhập thành công. Bạn có thể thêm `true` làm tham số thứ hai để luôn chuyển hướng đến URL này, bỏ qua URL gốc mà người dùng muốn truy cập.
- **failureUrl("/login?error")** : URL mà người dùng sẽ được chuyển hướng đến nếu đăng nhập thất bại.

- `usernameParameter("username")` / `passwordParameter("password")` : Tên của các trường input cho username và password trong form login.
- `logout()` : Cấu hình chức năng đăng xuất.
- `logoutUrl("/logout")` : URL để kích hoạt quá trình đăng xuất (mặc định là `POST /logout`).
- `logoutSuccessUrl("/login?logout")` : URL mà người dùng sẽ được chuyển hướng đến sau khi đăng xuất thành công.
- `invalidateHttpSession(true)` : Vô hiệu hóa session HTTP sau khi đăng xuất (mặc định là `true`).
- `deleteCookies("JSESSIONID")` : Xóa các cookie cụ thể sau khi đăng xuất.
- `csrf()` (**Cross-Site Request Forgery**): Spring Security cung cấp bảo vệ CSRF theo mặc định. Đối với các ứng dụng web truyền thống sử dụng session, việc bật CSRF là rất quan trọng. Tuy nhiên, đối với các RESTful API stateless (ví dụ: sử dụng JWT), bạn thường sẽ tắt nó (`.csrf(csrf -> csrf.disable())`) vì JWT tự nó đã cung cấp một mức độ bảo vệ chống lại CSRF.
- `httpBasic()` : Cấu hình xác thực HTTP Basic. Thường được sử dụng cho các API hoặc dịch vụ mà không có giao diện người dùng.
- `sessionManagement()` : Cấu hình quản lý session, bao gồm các chính sách tạo session, kiểm soát đồng thời session, v.v.

6.3.2. `UserDetailsService` và `PasswordEncoder`

Để Spring Security có thể xác thực người dùng, bạn cần cung cấp hai bean quan trọng:

- `UserDetailsService` : Interface này có một phương thức `loadUserByUsername(String username)` mà bạn cần triển khai để tải thông tin chi tiết về người dùng từ nguồn dữ liệu của bạn (ví dụ: database, LDAP). Phương thức này trả về một đối tượng `UserDetails` chứa thông tin về người dùng và các quyền của họ.
- `PasswordEncoder` : Interface này được sử dụng để mã hóa (encode) mật khẩu người dùng. Bạn **không bao giờ** nên lưu trữ mật khẩu dưới dạng văn bản thuần túy trong cơ sở

dữ liệu. Spring Security yêu cầu một `PasswordEncoder` để so sánh mật khẩu được cung cấp trong quá trình đăng nhập với mật khẩu đã mã hóa trong cơ sở dữ liệu.

6.3.3. AuthenticationManager

`AuthenticationManager` là interface chính trong Spring Security để thực hiện xác thực. Trong hầu hết các trường hợp, bạn không cần phải tự tạo bean này. Spring Boot sẽ tự động cấu hình một `AuthenticationManager` dựa trên `UserDetailsService` và `PasswordEncoder` mà bạn cung cấp.

Tuy nhiên, nếu bạn cần inject `AuthenticationManager` vào controller (ví dụ: để xử lý đăng nhập thủ công trong REST API), bạn có thể expose nó dưới dạng một bean:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.authentication.AuthenticationManager;
import
org.springframework.security.config.annotation.authentication.configuration.A
uthenticationConfiguration;

@Configuration
public class AuthenticationManagerConfig {

    @Bean
    public AuthenticationManager
authenticationManager(AuthenticationConfiguration
authenticationConfiguration) throws Exception {
        return authenticationConfiguration.getAuthenticationManager();
    }
}
```

Việc cấu hình Spring Security có thể trở nên phức tạp tùy thuộc vào yêu cầu bảo mật của ứng dụng. Tuy nhiên, với các công cụ và cách tiếp cận mà Spring Boot cung cấp, bạn có thể xây dựng một hệ thống bảo mật mạnh mẽ và linh hoạt một cách hiệu quả.

6.4. User Authentication (Xác thực người dùng)

User Authentication là quá trình xác minh danh tính của người dùng. Trong Spring Security, quá trình này được thực hiện bởi `AuthenticationManager` thông qua các `AuthenticationProvider` .

6.4.1. Luồng xác thực cơ bản

1. **Người dùng gửi thông tin đăng nhập:** Thông thường là username và password thông qua một form login hoặc JWT token trong header.
2. **`AuthenticationFilter` chặn request:** Một trong các `AuthenticationFilter` của Spring Security (ví dụ: `UsernamePasswordAuthenticationFilter` cho form login) sẽ chặn request và tạo ra một đối tượng `Authentication` chưa được xác thực (ví dụ: `UsernamePasswordAuthenticationToken`).
3. **`AuthenticationManager` xử lý:** Đối tượng `Authentication` này được chuyển đến `AuthenticationManager` .
4. **`AuthenticationManager` ủy quyền cho `AuthenticationProvider` :** `AuthenticationManager` (thường là `ProviderManager`) sẽ tìm kiếm một `AuthenticationProvider` phù hợp để xử lý loại `Authentication` cụ thể đó.
5. **`AuthenticationProvider` thực hiện xác thực:** `AuthenticationProvider` sẽ tải thông tin chi tiết về người dùng (thường thông qua `UserService`) và so sánh mật khẩu được cung cấp với mật khẩu đã lưu (sử dụng `PasswordEncoder`).
6. **Kết quả xác thực:**
 - Nếu xác thực thành công, `AuthenticationProvider` trả về một đối tượng `Authentication` đã được xác thực, chứa thông tin `UserDetails` và các `GrantedAuthority` của người dùng.
 - Nếu xác thực thất bại, một `AuthenticationException` sẽ được ném ra.
7. **Lưu trữ `Authentication` vào `SecurityContext` :** Nếu xác thực thành công, đối tượng `Authentication` đã được xác thực sẽ được lưu trữ vào `SecurityContextHolder` , làm cho thông tin người dùng có sẵn cho các request tiếp theo trong cùng một session.

6.4.2. `UserService` và `UserDetails`

- **UserDetailsService** : Đây là một interface quan trọng mà bạn cần triển khai để Spring Security biết cách tải thông tin người dùng từ nguồn dữ liệu của bạn (database, LDAP, v.v.). Phương thức chính là `loadUserByUsername(String username)` , trả về một đối tượng `UserDetails` .
- **UserDetails** : Interface này đại diện cho thông tin chi tiết về người dùng. Nó cung cấp các phương thức để lấy username, password, trạng thái tài khoản (enabled, locked, expired) và các quyền (`GrantedAuthority`). Spring Security sử dụng thông tin này để thực hiện xác thực và ủy quyền.

6.4.3. PasswordEncoder

`PasswordEncoder` là một interface được sử dụng để mã hóa và so sánh mật khẩu. Việc sử dụng `PasswordEncoder` là bắt buộc để bảo mật mật khẩu người dùng. Spring Security cung cấp nhiều triển khai khác nhau, phổ biến nhất là `BCryptPasswordEncoder` .

Java

```
public interface PasswordEncoder {
    String encode(CharSequence rawPassword);
    boolean matches(CharSequence rawPassword, String encodedPassword);
    default boolean upgradeEncoding(String encodedPassword) { /* ... */ }
}
```

Bạn cần khai báo một bean `PasswordEncoder` trong cấu hình bảo mật của mình:

Java

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

6.4.4. Các loại xác thực phổ biến

- **In-Memory Authentication:** Dùng cho mục đích phát triển hoặc các ứng dụng nhỏ, thông tin người dùng được lưu trữ trực tiếp trong bộ nhớ.

- **Database Authentication:** Thông tin người dùng được lưu trữ trong cơ sở dữ liệu. Đây là phương pháp phổ biến nhất cho các ứng dụng thực tế. Bạn sẽ triển khai `UserDetailsService` để tải người dùng từ DB.
- **LDAP Authentication:** Xác thực người dùng thông qua một máy chủ LDAP (Lightweight Directory Access Protocol).
- **OAuth2/OpenID Connect:** Xác thực thông qua các nhà cung cấp dịch vụ bên thứ ba như Google, Facebook, GitHub. Spring Security cung cấp hỗ trợ mạnh mẽ cho OAuth2 Client và Resource Server.
- **JWT (JSON Web Token) Authentication:** Một phương pháp xác thực stateless phổ biến cho RESTful APIs. Sau khi người dùng đăng nhập, server trả về một JWT. Client sẽ gửi JWT này trong mỗi request tiếp theo để xác thực. Spring Security không có hỗ trợ JWT tích hợp sẵn hoàn toàn, nhưng có thể dễ dàng tích hợp bằng cách thêm các filter và provider tùy chỉnh.

Quá trình xác thực là nền tảng của bảo mật ứng dụng, và Spring Security cung cấp một kiến trúc linh hoạt để bạn có thể lựa chọn và tùy chỉnh phương pháp xác thực phù hợp với yêu cầu của mình.

6.5. Authorization (Ủy quyền)

Authorization là quá trình xác định xem một người dùng đã được xác thực có quyền truy cập vào một tài nguyên cụ thể hoặc thực hiện một hành động cụ thể hay không. Trong Spring Security, ủy quyền có thể được thực hiện ở cấp độ URL (request-level authorization) hoặc cấp độ phương thức (method-level authorization).

6.5.1. Request-level Authorization (Ủy quyền cấp độ Request)

Đây là cách phổ biến nhất để bảo vệ các đường dẫn URL trong ứng dụng web. Bạn cấu hình các quy tắc ủy quyền trong `SecurityFilterChain` bằng cách sử dụng `authorizeHttpRequests()`.

Các biểu thức ủy quyền phổ biến:

- `permitAll()` : Cho phép truy cập mà không cần xác thực.

- **authenticated()** : Yêu cầu người dùng phải được xác thực (đã đăng nhập).
- **hasRole(String role)** : Yêu cầu người dùng có một vai trò cụ thể. Spring Security tự động thêm tiền tố `ROLE_` nếu bạn sử dụng `hasRole()` . Ví dụ, `hasRole("ADMIN")` sẽ kiểm tra quyền `ROLE_ADMIN` .
- **hasAnyRole(String... roles)** : Yêu cầu người dùng có ít nhất một trong các vai trò được chỉ định.
- **hasAuthority(String authority)** : Yêu cầu người dùng có một quyền cụ thể. Đây là cách linh hoạt hơn `hasRole()` vì bạn có thể định nghĩa các quyền chi tiết hơn (ví dụ: `DELETE_PRODUCT` , `VIEW_REPORT`).
- **hasAnyAuthority(String... authorities)** : Yêu cầu người dùng có ít nhất một trong các quyền được chỉ định.
- **access(String expression)** : Cho phép sử dụng Spring Expression Language (SpEL) để tạo các biểu thức ủy quyền phức tạp hơn. Các biểu thức này có thể truy cập vào `Authentication` object, các thuộc tính của request, và các bean trong Spring context.

Ví dụ cấu hình:

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.web.builders.HttpSecurity;
import
org.springframework.security.config.annotation.web.configuration.EnableWebSec
urity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
@EnableWebSecurity
public class WebSecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
```

```

        .requestMatchers("/public/**").permitAll() // Public access
        .requestMatchers("/admin/**").hasRole("ADMIN") // Only ADMIN
role
        .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN") //
USER or ADMIN role

        .requestMatchers("/api/products/delete").hasAuthority("PRODUCT_DELETE") //
Specific authority
        .anyRequest().authenticated() // All other requests require
authentication
    )
    .formLogin(form -> form
        .loginPage("/login")
        .permitAll()
    )
    .logout(logout -> logout.permitAll());
return http.build();
}
}

```

6.5.2. Method-level Authorization (Ủy quyền cấp độ Phương thức)

Ngoài việc bảo vệ các URL, Spring Security còn cho phép bạn bảo vệ các phương thức cụ thể trong các lớp Service hoặc Repository. Điều này rất hữu ích khi bạn muốn kiểm soát quyền truy cập dựa trên logic nghiệp vụ phức tạp hoặc khi một phương thức có thể được gọi từ nhiều nơi khác nhau.

Để kích hoạt method-level security, bạn cần thêm annotation `@EnableMethodSecurity` vào lớp cấu hình của mình:

Java

```

import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity;

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true,
jsr250Enabled = true)
public class MethodSecurityConfig {
    // Cấu hình khác nếu cần
}

```

- `prePostEnabled = true` : Kích hoạt các annotation `@PreAuthorize` và `@PostAuthorize` .
- `securedEnabled = true` : Kích hoạt annotation `@Secured` .
- `jsr250Enabled = true` : Kích hoạt các annotation chuẩn JSR-250 như `@RolesAllowed` .

Các Annotation phổ biến:

- `@PreAuthorize(String expression)` : Biểu thức SpEL được đánh giá **trước khi** phương thức được thực thi. Nếu biểu thức trả về `false` , phương thức sẽ không được gọi và một `AccessDeniedException` sẽ được ném ra.
- `@PostAuthorize(String expression)` : Biểu thức SpEL được đánh giá **sau khi** phương thức được thực thi, nhưng trước khi kết quả được trả về cho người gọi. Điều này hữu ích khi bạn cần kiểm tra quyền dựa trên giá trị trả về của phương thức.
- `@Secured(String... roles)` : Một annotation đơn giản hơn, chỉ chấp nhận danh sách các vai trò (roles). Người dùng phải có ít nhất một trong các vai trò được liệt kê để truy cập phương thức.
- `@RolesAllowed(String... roles)` (**JSR-250**): Tương tự như `@Secured` , nhưng là một phần của chuẩn JSR-250. Cũng chấp nhận danh sách các vai trò.

Ví dụ về Method Security:

Java

```
import org.springframework.security.access.prepost.PreAuthorize;
import org.springframework.security.access.prepost.PostAuthorize;
import org.springframework.stereotype.Service;

@Service
public class ProductService {

    @PreAuthorize("hasRole(\"ADMIN\")")
    public void deleteProduct(Long id) {
        System.out.println("Deleting product with ID: " + id);
    }

    @PreAuthorize("hasAnyRole(\"USER\", \"ADMIN\")")
    public Product getProductDetails(Long id) {
        // Logic lấy sản phẩm
        return new Product(id, "Sample Product", 100.0);
    }
}
```

```
}

@PostAuthorize("returnObject.owner == authentication.name")
public Order getOrderForCurrentUser(Long orderId) {
    // Logic lấy đơn hàng
    Order order = new Order(orderId, "someUser"); // Giả sử owner là
    "someUser"
    return order;
}
}
```

6.5.3. Quyết định sử dụng Request-level hay Method-level Authorization

- **Request-level Authorization:** Phù hợp cho việc bảo vệ các URL chung, các tài nguyên tĩnh, hoặc khi quyền truy cập chỉ phụ thuộc vào đường dẫn và phương thức HTTP.
- **Method-level Authorization:** Phù hợp khi quyền truy cập phụ thuộc vào dữ liệu cụ thể (ví dụ: chỉ chủ sở hữu mới được chỉnh sửa tài nguyên), hoặc khi một phương thức có thể được gọi từ nhiều ngữ cảnh khác nhau và cần kiểm soát quyền truy cập chi tiết hơn.

Trong nhiều ứng dụng, bạn sẽ sử dụng kết hợp cả hai để đạt được mức độ bảo mật mong muốn.

6.6. Authorization Filter (Bộ lọc Ủy quyền)

Trong Spring Security, ủy quyền (Authorization) được thực hiện thông qua một chuỗi các bộ lọc (Filter Chain). Khi một request HTTP đến ứng dụng, nó sẽ đi qua một loạt các bộ lọc trước khi đến được DispatcherServlet. Các bộ lọc này có nhiệm vụ khác nhau, từ xác thực, quản lý session, đến ủy quyền.

6.6.1. Vai trò của Filter trong Spring Security

Spring Security được xây dựng dựa trên kiến trúc Servlet Filter. Mỗi `Filter` trong chuỗi có một trách nhiệm cụ thể. Khi một request đến, nó sẽ được xử lý tuần tự bởi các `Filter` này. Nếu một `Filter` quyết định rằng request không được phép tiếp tục (ví dụ: người dùng chưa được xác thực hoặc không có quyền), nó có thể chặn request và trả về một phản hồi lỗi.

6.6.2. Các Filter liên quan đến Authorization

Trong chuỗi Filter của Spring Security, có một số Filter đóng vai trò quan trọng trong quá trình ủy quyền:

1. **FilterSecurityInterceptor** : Đây là Filter cuối cùng trong chuỗi Filter của Spring Security và là nơi diễn ra quá trình ủy quyền chính. `FilterSecurityInterceptor` sẽ:
 - Lấy thông tin xác thực của người dùng từ `SecurityContextHolder` .
 - Xác định tài nguyên được yêu cầu (ví dụ: URL, phương thức HTTP).
 - Tìm kiếm các cấu hình bảo mật (ví dụ: `hasRole('ADMIN')` , `permitAll()`) áp dụng cho tài nguyên đó.
 - Sử dụng `AccessDecisionManager` để đưa ra quyết định cuối cùng về việc cho phép hay từ chối truy cập.
 - Nếu truy cập bị từ chối, nó sẽ ném ra một `AccessDeniedException` .
2. **ExceptionTranslationFilter** : Filter này nằm trước `FilterSecurityInterceptor` trong chuỗi. Nhiệm vụ của nó là bắt các ngoại lệ liên quan đến bảo mật (như `AuthenticationException` và `AccessDeniedException`) và chuyển đổi chúng thành các phản hồi HTTP phù hợp.
 - Nếu bắt được `AuthenticationException` (do xác thực thất bại), nó sẽ chuyển hướng người dùng đến trang đăng nhập hoặc trả về lỗi 401 Unauthorized.
 - Nếu bắt được `AccessDeniedException` (do ủy quyền thất bại), nó sẽ chuyển hướng người dùng đến trang lỗi 403 Forbidden hoặc trả về lỗi 403 Forbidden.

6.6.3. Luồng Authorization với Filter

Khi một request đến, luồng xử lý ủy quyền trong Spring Security diễn ra như sau:

1. **Request đi qua các Authentication Filters:** Các Filter này cố gắng xác thực người dùng. Nếu thành công, thông tin `Authentication` sẽ được lưu vào `SecurityContextHolder` .
2. **Request đến FilterSecurityInterceptor** : Filter này kiểm tra xem người dùng có quyền truy cập vào tài nguyên được yêu cầu hay không dựa trên các quy tắc cấu hình (`authorizeHttpRequests()`).

3. **AccessDecisionManager đưa ra quyết định:** `FilterSecurityInterceptor` ủy quyền cho `AccessDecisionManager` để đưa ra quyết định ủy quyền cuối cùng. `AccessDecisionManager` sẽ tham khảo các `AccessDecisionVoter` để bỏ phiếu.

4. Kết quả ủy quyền:

- Nếu được phép, request tiếp tục đi đến `DispatcherServlet` và được xử lý bởi `Controller`.
- Nếu bị từ chối, `FilterSecurityInterceptor` ném ra `AccessDeniedException`.

5. **ExceptionHandler xử lý ngoại lệ:** `ExceptionHandler` bắt `AccessDeniedException` và xử lý nó bằng cách trả về lỗi 403 Forbidden hoặc chuyển hướng đến trang lỗi.

6.6.4. Tùy chỉnh Filter Chain

Bạn có thể tùy chỉnh chuỗi Filter của Spring Security bằng cách thêm các Filter của riêng mình hoặc thay đổi thứ tự của các Filter hiện có. Điều này thường được thực hiện thông qua `HttpSecurity` trong lớp cấu hình bảo mật.

Ví dụ: Thêm một Custom Filter

Java

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
```

```

    public SecurityFilterChain securityFilterChain(HttpSecurity http,
MyCustomFilter myCustomFilter) throws Exception {
        http
            // ... các cấu hình khác
            .addFilterBefore(myCustomFilter,
UsernamePasswordAuthenticationFilter.class); // Thêm custom filter trước
UsernamePasswordAuthenticationFilter

        return http.build();
    }

    // Định nghĩa MyCustomFilter (ví dụ)
    // @Component
    // public class MyCustomFilter extends OncePerRequestFilter {
    //     @Override
    //     protected void doFilterInternal(HttpServletRequest request,
HttpServletRequest response, FilterChain filterChain) throws
ServletException, IOException {
        //         // Logic của custom filter
        //         filterChain.doFilter(request, response);
        //     }
    // }
}

```

Việc hiểu rõ cách các Filter hoạt động và tương tác với nhau là chìa khóa để tùy chỉnh và gỡ lỗi các vấn đề bảo mật trong ứng dụng Spring Boot của bạn.

6.7. Method Security (Bảo mật cấp độ Phương thức)

Ngoài việc bảo vệ các URL (request-level security), Spring Security còn cho phép bạn bảo vệ các phương thức cụ thể trong các lớp Service hoặc Repository. Điều này rất hữu ích khi bạn muốn kiểm soát quyền truy cập dựa trên logic nghiệp vụ phức tạp, hoặc khi một phương thức có thể được gọi từ nhiều nơi khác nhau và cần kiểm soát quyền truy cập chi tiết hơn.

6.7.1. Kích hoạt Method Security

Để sử dụng method security, bạn cần kích hoạt nó trong lớp cấu hình Spring Security của mình bằng cách sử dụng annotation `@EnableMethodSecurity` (hoặc

`@EnableGlobalMethodSecurity` cho các phiên bản Spring Security cũ hơn).

```
import org.springframework.context.annotation.Configuration;
import
org.springframework.security.config.annotation.method.configuration.EnableMet
hodSecurity;

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true,
jsr250Enabled = true)
public class MethodSecurityConfig {
    // Cấu hình khác nếu cần
}
```

- **prePostEnabled = true** : Kích hoạt các annotation `@PreAuthorize` và `@PostAuthorize` . Đây là các annotation mạnh mẽ nhất vì chúng cho phép sử dụng Spring Expression Language (SpEL) để định nghĩa các quy tắc bảo mật phức tạp.
- **securedEnabled = true** : Kích hoạt annotation `@Secured` . Đây là một annotation đơn giản hơn, chỉ chấp nhận danh sách các vai trò (roles).
- **jsr250Enabled = true** : Kích hoạt các annotation chuẩn JSR-250 như `@RolesAllowed` . Tương tự như `@Secured` nhưng là một phần của chuẩn Java EE.

6.7.2. Các Annotation phổ biến cho Method Security

Spring Security cung cấp một số annotation để áp dụng bảo mật ở cấp độ phương thức:

1. `@PreAuthorize(String expression)`

- **Mô tả:** Biểu thức SpEL được đánh giá **trước khi** phương thức được thực thi. Nếu biểu thức trả về `false` , phương thức sẽ không được gọi và một `AccessDeniedException` sẽ được ném ra.
- **Khi sử dụng:** Khi bạn cần kiểm tra quyền truy cập dựa trên các điều kiện phức tạp, chẳng hạn như kiểm tra vai trò, quyền, hoặc thậm chí là dữ liệu đầu vào của phương thức.
- **Ví dụ:**

2. `@PostAuthorize(String expression)`

- **Mô tả:** Biểu thức SpEL được đánh giá **sau khi** phương thức được thực thi, nhưng trước khi kết quả được trả về cho người gọi. Điều này hữu ích khi bạn cần kiểm tra quyền dựa trên giá trị trả về của phương thức.
- **Khi sử dụng:** Khi quyết định ủy quyền phụ thuộc vào dữ liệu được trả về bởi phương thức. Ví dụ, chỉ cho phép người dùng xem tài liệu nếu họ là chủ sở hữu của tài liệu đó.
- **Ví dụ:**

3. `@Secured(String... roles)`

- **Mô tả:** Một annotation đơn giản hơn, chỉ chấp nhận danh sách các vai trò (roles). Người dùng phải có ít nhất một trong các vai trò được liệt kê để truy cập phương thức.
- **Khi sử dụng:** Khi quy tắc bảo mật chỉ đơn giản là kiểm tra vai trò của người dùng.
- **Ví dụ:**

4. `@RolesAllowed(String... roles)` (JSR-250)

- **Mô tả:** Tương tự như `@Secured`, nhưng là một phần của chuẩn JSR-250 (Java Specification Request 250). Cũng chấp nhận danh sách các vai trò.
- **Khi sử dụng:** Nếu bạn muốn sử dụng các annotation bảo mật chuẩn Java EE thay vì các annotation cụ thể của Spring Security.
- **Ví dụ:**

6.7.3. Spring Expression Language (SpEL) trong Method Security

SpEL là một ngôn ngữ biểu thức mạnh mẽ được Spring sử dụng để tạo các quy tắc bảo mật phức tạp. Trong `@PreAuthorize` và `@PostAuthorize`, bạn có thể sử dụng các đối tượng và hàm sau:

- **authentication**: Đại diện cho đối tượng `Authentication` hiện tại, chứa thông tin về người dùng đã xác thực.
- **authentication.principal**: Đối tượng `UserDetails` của người dùng.
- **authentication.name**: Username của người dùng.

- `authentication.authorities` : Danh sách các quyền của người dùng.
- `hasRole(String role)` : Kiểm tra xem người dùng có vai trò cụ thể hay không.
- `hasAnyRole(String... roles)` : Kiểm tra xem người dùng có bất kỳ vai trò nào trong danh sách hay không.
- `hasAuthority(String authority)` : Kiểm tra xem người dùng có quyền cụ thể hay không.
- `hasAnyAuthority(String... authorities)` : Kiểm tra xem người dùng có bất kỳ quyền nào trong danh sách hay không.
- `#parameterName` : Truy cập các tham số của phương thức. Ví dụ: `#userId` .
- `returnObject` : Trong `@PostAuthorize` , đại diện cho giá trị trả về của phương thức.

6.7.4. Khi nào sử dụng Method Security?

- **Kiểm soát truy cập chi tiết:** Khi bạn cần kiểm soát quyền truy cập ở mức độ hạt nhân (granular level), ví dụ, chỉ cho phép người dùng chỉnh sửa tài nguyên của chính họ.
- **Logic nghiệp vụ phức tạp:** Khi quyết định ủy quyền phụ thuộc vào logic nghiệp vụ phức tạp hoặc dữ liệu cụ thể liên quan đến phương thức.
- **Tái sử dụng logic bảo mật:** Khi một phương thức được gọi từ nhiều nơi khác nhau trong ứng dụng và bạn muốn đảm bảo rằng logic bảo mật được áp dụng nhất quán.

Method security bổ sung một lớp bảo vệ mạnh mẽ cho ứng dụng của bạn, cho phép bạn định nghĩa các quy tắc ủy quyền chính xác và linh hoạt hơn so với chỉ bảo vệ ở cấp độ URL.